Module 3.2 — Cloud Infrastructure and Platforms in Industry

Amazon Web Services (AWS): Compute, Storage, Communication & Additional Services

> **Course Notes** | Prof. Shweta Dhawan Chachra | Date: 02-03-2026

PART 1: Introduction to AWS and Its Architecture

1.1 Public Clouds and Service Offerings

Cloud computing providers offer public cloud services that allow organizations and individuals to access computing resources over the Internet on a pay-as-you-go basis. These services remove the need to own and maintain physical infrastructure.

1.2 Amazon Web Services (AWS) — Overview

Definition

Amazon Web Services (AWS) is a public cloud computing platform offered by Amazon. It is accessible at http://aws.amazon.com/.

Key Points

- **Virtual Machines (VMs)** are at the heart of AWS. VMs enable the sharing of computing resources **flexibly** and **safely** across multiple users.
- Amazon has been one of the **pioneering leaders** in providing public cloud services.
- AWS operates under the **IaaS (Infrastructure as a Service)** model, meaning Amazon provides the underlying hardware and virtualization infrastructure, and users manage the software and applications on top of it.

1.3 AWS Architecture

Diagram Description

The AWS Architecture diagram illustrates how the core services of AWS are connected and interact with each other. The major components are:

Components and Their Functions

Component	Full Name	Function
EC2	Elastic Compute Cloud	Provides virtualized platforms (VMs) on which cloud applications run
S3	Simple Storage Service	Provides object-oriented (distributed) storage for users
EBS	Elastic Block Service	Provides block storage interface for traditional applications
SQS	Simple Queue Service	Provides reliable message delivery between two processes

Explanation of Each Component

1. EC2 (Elastic Compute Cloud) EC2 is the core compute service of AWS. It provides virtualized server environments (called instances) where users can run any application. These instances are essentially virtual machines hosted on Amazon's physical servers.

2. S3 (Simple Storage Service) S3 is a distributed object storage service. It allows users to store and retrieve data in any format (files, images, backups, etc.) from anywhere on the Internet.

3. EBS (Elastic Block Service) EBS provides block-level storage, similar to a traditional hard disk. It is primarily used to support applications that require persistent storage and a standard file system interface.

4. SQS (Simple Queue Service) SQS ensures **reliable message communication** between two independent processes. Importantly, messages stored in the queue are preserved even if the **receiver process is not currently running**, making communication asynchronous and fault-tolerant.

5. SOAP Access Users can access their stored objects through **SOAP (Simple Object Access Protocol)** using web browsers or any client programs that support the SOAP standard.

PART 2: Compute Services

2.1 Introduction to Compute Services

Definition

Compute services are the **fundamental element** of cloud computing systems. They provide the processing power needed to run applications on the cloud.

Key Point

The primary compute service on AWS is **Amazon EC2 (Elastic Compute Cloud)**, which provides

an IaaS solution that has become a **reference model** for similar offerings from other cloud vendors.

2.2 Amazon EC2 — Elastic Compute Cloud

What EC2 Does

- EC2 allows users to **deploy servers in the form of virtual machines (VMs)**, created as instances of a specific image (AMI).
- These images come preinstalled with an **operating system** and a **software stack**.
- Instances are configurable for **memory, number of processors, and storage**.
- Users are given **credentials** to remotely access and configure their instances or install additional software.

2.3 Amazon Machine Images (AMIs)

Definition

Amazon Machine Images (AMIs) are **templates** used to create virtual machines (EC2 instances). Think of them as a snapshot or blueprint of a configured server environment.

Storage and Identification

- AMIs are stored in **Amazon S3**.
- Each AMI is identified by a **unique identifier** in the format: `ami-xxxxxx`
- AMIs also have an associated **manifest XML file** that describes the image.

Contents of an AMI

An AMI contains:

- A **physical file system layout** with a predefined operating system installed.
- Two additional components specified during AMI configuration:
  - **ARI (Amazon Ramdisk Image)** — identified by `ari-yyyyyy`
  - **AKI (Amazon Kernel Image)** — identified by `aki-zzzzzz`

Creating a New AMI (Step-by-Step Process)

1. Start from an existing prebuilt AMI.
2. Launch an EC2 instance using that AMI.
3. Log into the running instance.

4. Install required software and customize the environment as needed.
5. Save this updated configuration as a **new custom AMI**.
6. All future instances launched from this new AMI will include your customizations.
7. The resulting AMI is stored in an **S3 bucket**.

Sharing and Privacy Options

Once an AMI is created, the owner can:

- **Keep it private** — only the owner can use it.
- **Share it with specific AWS accounts** — a selected list of users.
- **Make it public** — available to the entire AWS community.
- **Sell it** — the owner can associate a **product code** with the AMI and earn **revenue** every time someone uses it to create EC2 instances.

> **Reference:** AWS AMI Documentation

2.4 EC2 Instances

Definition

EC2 instances are **virtual machines** created using AMIs as templates. They are customized by selecting the number of cores, computing power, and installed memory.

EC2 Compute Units (ECU)

- **ECU** is a measure of the **computing power of a virtual core**.
- It expresses a **predictable, standardized quantity of real CPU power** allocated to an instance.
- Instead of using actual CPU frequencies (which change over time), Amazon uses ECUs so that instance performance remains **consistent** with industry standards regardless of changes in underlying hardware.

2.5 Six Major Categories of EC2 Instances

1. Standard Instances

- Suitable for **most general-purpose applications**.
- EC2 provides **three sub-categories** of increasing computing power, storage, and memory.

2. Micro Instances

- Designed for applications that use **limited CPU and memory**.
- Can handle **occasional bursts** in CPU cycles to process workload spikes.
- Example use case: **Small web applications with limited traffic**.

3. High-Memory Instances

- Targets applications requiring **large amounts of memory**.
- Ideal for **three-tier web applications** with high traffic.
- Three sub-categories are available; **memory grows proportionally more** than CPU.

4. High-CPU Instances

- Targets **compute-intensive applications** where processing power matters more than memory.
- Two configurations available where **computing power grows proportionally more** than memory.

5. Cluster Compute Instances

- Used to provide **virtual cluster services**.
- Features: **High CPU, large memory, extremely high I/O and network performance**.
- Suitable for **HPC (High Performance Computing)** applications.

6. Cluster GPU Instances

- Features **Graphic Processing Units (GPUs)** along with high CPU, large memory, and high I/O.
- Ideal for **heavy graphic computations** such as rendering clusters.
- Since GPUs can also perform general-purpose computing, these instances are also suitable for **HPC applications**.

2.6 EC2 Pricing

- EC2 instances are priced **hourly** based on the instance category.
- At the **beginning of every hour of usage**, the full hour's cost is charged.
- The hourly rate for each instance type is constant (does not fluctuate mid-hour).

2.7 Running EC2 Instances

Instances can be launched in two ways:

1. **Command-line tools** provided by Amazon — connects to the AWS Web Service for remote access to EC2.
2. **AWS Console (web interface)** — allows management of EC2 and other services such as S3 through a graphical browser-based dashboard.

2.8 EC2 Environment

The EC2 environment manages the **operational surroundings** of each virtual machine. It is responsible for:

- **Allocating IP addresses** to instances.
- **Attaching storage volumes** to instances.
- **Configuring security** — access control and network connectivity.

Networking in EC2

Internal IP Address: By default, all EC2 instances are assigned an **internal IP address**, which allows them to communicate within the EC2 network and access the Internet as clients.

Elastic IP Address:

- A **static, public IPv4 address** that is associated with your AWS account in a specific Region.
- Unlike auto-assigned public IPs (which change every time an instance is stopped and started), an **Elastic IP is preserved** even after stop/start cycles.
- Elastic IPs can be **remapped to different instances**, enabling:
  - Instances to act as **publicly accessible servers**.
  - **Failover capabilities** (if one instance fails, the IP is reassigned to a backup instance).

> **Key FAQ:** If your EC2 instance's public IP changes every time you stop/start it, assign an **Elastic IP** to get a stable, static address.

Domain Name Format: Each EC2 instance is also given a domain name in the format:

```
ec2-xx-xx-xx-xx.compute-x.amazonaws.com
```

Where:

- `xx-xx-xx-xx` = four parts of the external IP address separated by dashes.

- `compute-x` = indicates the **availability zone** (e.g., `compute-1` for us-east-1, `compute-2` for us-west-1).

Example: `ec2-54-12-34-56.compute-1.amazonaws.com`

2.9 EC2 Availability Zones

AWS has multiple availability zones, each priced differently:

- **United States:** Virginia, Northern California
- **Europe:** Ireland
- **Asia Pacific:** Singapore, Tokyo

Instance owners have **partial control** over where their instances are deployed.

2.10 EC2 Security

Key Pair Access

- Owners can associate a **key pair** with one or more instances at the time of creation.
- This key pair allows the owner to **remotely connect** to the running instance and gain **root access**.

Firewall and Security Groups

- Amazon EC2 uses **basic firewall configuration** to control instance accessibility.
- Firewall rules allow specification of:
  - **Source address**
  - **Port**
  - **Protocols** (TCP, UDP, ICMP)
- **Security groups** can have rules attached to them. Instances can be part of **one or more security groups** before deployment.
- Security groups provide **flexible basic security**, but must be **complemented with security configuration inside the instance itself**.

Definition

Amazon Elastic MapReduce provides AWS users with a **cloud computing platform for MapReduce applications** — a programming model used for processing large datasets in parallel.

Key Points

- Uses **Hadoop** as the MapReduce engine.
- Deployed on a **virtual infrastructure of EC2 instances**.
- Uses **Amazon S3** for storage needs.
- Ideal for big data processing tasks.

PART 3: Storage Services

3.1 Overview of AWS Storage Services

AWS provides a **collection of storage services** catering to different needs. The core service is **Amazon Simple Storage Service (S3)**.

3.2 Amazon S3 — Simple Storage Service

Definition

S3 is a **distributed object store** that allows users to store information in different formats. It is accessible through a **REST (Representational State Transfer)** interface — similar to a distributed file system but with key differences that make the infrastructure highly efficient.

3.3 S3 Key Concepts (Four Design Principles)

1. The Storage is Organized in a Two-Level Hierarchy (Buckets & Objects)

- S3 organizes storage into **buckets** which cannot be further partitioned.
- It is **not possible to create directories** or physical groupings within a bucket.

2. Stored Objects Cannot Be Manipulated Like Standard Files

- S3 is designed for storing **objects that will not change over time**.
- The following operations are **NOT allowed** once an object is stored:
    - Renaming

    - Modifying
    - Relocating
- To "change" an object, you must **delete it and re-add it**.
- Once stored, an object's content and position are **immutable**.

3. Content Is Not Immediately Available to Users (Eventual Consistency)

- S3 is designed as an **eventually consistent data store**.
- Because it is a **large distributed storage facility**, changes are not immediately reflected globally.
- S3 uses **replication** for redundancy and efficient global delivery — this introduces **latency** especially for large objects.
- An object uploaded to S3 may **not be instantly available** across the entire globe.

4. Requests Will Occasionally Fail

- Due to the large distributed infrastructure, requests may occasionally fail.
- Under certain conditions, S3 returns an **internal server error** (drops the request).
- A **small failure rate** is expected during normal operations — this is **not a persistent failure** but a known behavior.

3.4 Core Components of S3: Buckets and Objects

Buckets

Definition: A bucket is a **virtual container** for storing objects. It functions like a **virtual drive** providing a flat (non-hierarchical) store.

Properties:

- Buckets are **top-level elements** in S3 and do **not support nesting**.
- Located in a **specific geographic location** and replicated for fault tolerance.
- By default, buckets are created in **Amazon's U.S. datacenters**.
- Once created, a bucket **cannot be renamed or relocated** — it must be deleted and recreated.
- Deletion of a bucket requires the bucket to be **empty**.
- All objects in a bucket are stored in the **same availability zone** as the bucket.

Accessing Buckets via REST: Buckets are accessible via URIs under the `s3.amazonaws.com` domain.

REST Operations on Buckets:

Operation	HTTP Method
Create a bucket	PUT request to `http://s3.amazonaws.com/` with bucket name
List contents	GET request with bucket name
Delete a bucket	DELETE request (only if bucket is empty)

Objects

Definition: Objects are the **actual content elements** stored in S3. An object is like a file stored within a bucket.

Properties:

- Each object has a **unique name within its bucket**.
- Object name can be up to **1,024 bytes** (encoded in UTF-8).
- Almost **any character** is allowed in object names (even path separators like `/`), since buckets don't support directories. This allows **emulating directories** through naming conventions.
- Maximum object size: **5 GB**.
- Once created, an object **cannot be modified, renamed, or moved** to another bucket.

REST Operations on Objects:

Operation	HTTP Method
Create an object	PUT request with bucket name, object name, contents
Retrieve an object	GET request
Delete an object	DELETE request

Metadata

- Objects can be **tagged with metadata** passed as properties in the PUT request.
- Metadata can be retrieved via:

- **GET request** — returns both content and metadata.
- **HEAD request** — returns only metadata (without the content). Useful for lightweight checks.

3.5 Access Control and Security in S3

Access Control Policies (ACPs)

Amazon S3 controls access to buckets and objects using **Access Control Policies (ACPs)**.

- An ACP is a **set of grant permissions** attached to a resource.
- ACPs are expressed as **XML configuration files**.

Five Permission Types

Permission	What It Allows
READ	Retrieve an object and its metadata; list a bucket's content and get its metadata
WRITE	Add an object to a bucket or remove it
READ_ACP	Read the ACP (Access Control Policy) of a resource
WRITE_ACP	Modify the ACP of a resource
FULL_CONTROL	Grants all of the above permissions

3.6 Amazon Elastic Block Store (EBS)

Definition

Amazon EBS provides EC2 instances with **persistent storage** in the form of **volumes** that can be mounted at instance startup.

Key Features

- Accommodates up to **1 TB of space** per volume.
- Accessed through a **block device interface**, similar to a physical hard drive.
- Can be formatted with **any file system** (raw storage, NTFS, ext4, etc.).
- The content of an EBS volume **survives the instance lifecycle** — data persists even after the EC2 instance is terminated.
- Content is **persisted into S3** for durability.

- EBS volumes can be:
  - **Cloned** (copied)
  - Used as **boot partitions**
  - **Snapshotted** incrementally (incremental backups)

3.7 S3 vs. EBS — Detailed Comparison

Accessibility

S3	EBS
Accessible via HTTP/HTTPS from anywhere	Only accessible when mounted to an EC2 instance
Can be accessed by multiple users via web browsers, HTTP clients, even BitTorrent	Data accessible only by the instance to which it is connected
Used by many websites to host files	Not directly accessible from the Internet

Storage Type

S3	EBS
Object storage — for large numbers of files, backups, media	**Block storage** — like a hard disk attached to a virtual machine
Files stored in an unstructured manner	Appears as a hard disk partition to the EC2 instance
Accessed over network via API	Accessed like local storage

Size of Data

S3	EBS
Standard limit: **100 buckets**, each with **unlimited capacity**	Standard limit: **20 volumes**, each holding up to **1 TB**
Effectively unlimited data storage	Has an upper limit on storage per volume

Limitations of EBS

- EBS volumes **cannot be shared among multiple instances simultaneously** — once

mounted to one instance, no other instance can use it.

- However, **multiple volumes can be attached to a single instance**.

Pros and Cons of S3

Pros:

- Supports **multiple simultaneous readers** through replication.
- Accessible from anywhere.
- Good for large-scale storage.

Cons (Eventual Consistency):

- Changes are **not written immediately**.
- If you write something, a subsequent read might return **stale data** — this is the "eventual consistency" side effect.

EBS Advantage: Data reads and writes occur **almost instantly** (low latency).

Security

S3	EBS
Access management tools + encryption policies	No inherent security mechanism
Can prevent unauthorized access	If an attacker gains EC2 access, they can access the EBS volume

Application Use Cases

S3	EBS
Website image/video hosting, data analytics, IoT, big data, archive, backup, mobile apps	Business continuity, transactional databases, NoSQL databases, software testing

---

3.8 Amazon ElastiCache

Definition

Amazon ElastiCache is an **elastic, in-memory caching service** built on a cluster of EC2 instances running caching software, made available through web services.

Key Features

- Supports **automatic patch management**, **failure detection**, and **recovery** of cache nodes — all without administrative intervention from users.
- AWS users only need to **elastically resize** (scale up or down) the cluster when needed.
- An ElastiCache cluster can be **dynamically resized** based on demand from client applications.

Pricing

- ElastiCache nodes are priced similarly to the **EC2 costing model**, with a small additional cost for the caching service.
- Multiple instance types are available to choose from based on your requirements.

PART 4: Structured Storage Solutions

4.1 Introduction

Context

Enterprise applications often require **structured data storage** with capabilities for indexing and analytics. Traditionally, **RDBMS (Relational Database Management Systems)** have been the standard backend for such applications.

AWS Structured Storage Options

Amazon provides three forms of structured storage:

1. **Preconfigured EC2 AMIs**
2. **Amazon RDS (Relational Data Storage)**
3. **Amazon SimpleDB**

4.2 Preconfigured EC2 AMIs

Definition

Preconfigured EC2 AMIs are **predefined templates** with a database management system already installed. Users can launch EC2 instances from these AMIs and attach an EBS volume for persistent storage.

Available DBMS Templates

- IBM DB2

- Microsoft SQL Server
- MySQL
- Oracle
- PostgreSQL
- Sybase
- Vertica

Pricing

Instances are priced **hourly** based on the EC2 cost model.

Limitation

This approach places the **greatest administrative burden on the user**, who must configure, maintain, and manage the relational database software. However, it offers the **greatest variety of database products** to choose from.

4.3 Amazon RDS — Relational Database Service

Definition

Amazon RDS is a **fully managed relational database service** built on EC2 infrastructure. Amazon handles the database management tasks on behalf of users.

Key Advantages Over Preconfigured AMIs

- Users are **not responsible** for managing, configuring, or patching the database software — AWS handles all of this.
- Provides **automatic backups**, snapshots, and point-in-time recoveries.
- Includes **replication facilities** for high availability.
- Services are accessible via **AWS Console** or a dedicated Web service.

> *"Amazon RDS is a managed database service. It's responsible for most management tasks. By eliminating tedious manual tasks, Amazon RDS frees you to focus on your application and your users."* — Amazon AWS

Supported Database Engines

- MariaDB
- Microsoft SQL Server
- MySQL
- Oracle
- PostgreSQL

Specific Advantages of Amazon RDS

1. Familiar Database Products Users can work with the database engines they already know — no need to learn proprietary systems.

2. Automated Management RDS manages backups, software patching, automatic failure detection, and recovery. Users can also manually create backup snapshots and restore from them at any time.

3. High Availability

- Offers a **primary instance** and a **synchronous secondary instance** for failover.
- **Read replicas** can be added to increase read performance.

4. Security

- Integrated with **AWS IAM (Identity and Access Management)** to define users and permissions.
- Databases can be placed inside a **Virtual Private Cloud (VPC)** for network-level isolation.

4.4 DB Instances in RDS

Definition

A **DB Instance** is the basic building block of Amazon RDS — an isolated database environment in the AWS Cloud.

Properties

- Can contain **one or more user-created databases**.
- Can be accessed using the **same tools** you'd use with a standalone database.
- Can be created and modified using:
  - **AWS Command Line Interface (CLI)**
  - **Amazon RDS API**
  - **AWS Management Console**

4.5 DB Engines in RDS

A **DB Engine** is the specific relational database software running on a DB instance.

Each engine:

- Has its own **supported features** per version.

- Uses a **DB parameter group** to control the behavior of the databases it manages.

Currently supported engines:

- MariaDB, Microsoft SQL Server, MySQL, Oracle, PostgreSQL

4.6 DB Instance Types

Amazon RDS supports the following DB instance types:

1. **General-purpose** — Balanced CPU, memory, and network.
2. **Memory-optimized** — For memory-intensive database workloads.
3. **Burstable-performance** — For workloads with variable CPU needs.
4. **Optimized Reads** — For read-heavy database workloads.

4.7 DB Instance Classes

The **DB instance class** determines the **computation and memory capacity** of an RDS instance.

Format: `db.[instance_type].[size]`

Example 1:

- `db.m6g` = General-purpose instance powered by AWS Graviton2 processors.
- `db.m6g.2xlarge` = A specific class within the `db.m6g` type.

Example 2:

- `db.r6g` = Memory-optimized instance class powered by AWS Graviton2 processors.
- `db.r6g.2xlarge` = A specific class within this type.

General-Purpose DB Instance Types (db.m series)

`db.m7g`, `db.m6g`, `db.m6i`, `db.m5`, `db.m4`, `db.m3`

Memory-Optimized DB Instance Types (db.r and db.x series)

`db.x2g`, `db.z1d`, `db.x2i`, `db.x1e`, `db.x1`, `db.r6g`, `db.r6gd`, `db.r6i`, `db.r5b`, `db.r5d`, `db.r5`, `db.r4`, `db.r3`

Burstable-Performance DB Instance Types (db.t series)

`db.t4g`, `db.t3`, `db.t2`

4.8 Amazon DynamoDB

Definition

Amazon DynamoDB is a **fully managed NoSQL database service** designed to provide **fast and predictable performance** with seamless scalability.

Key Features

- Offloads administrative burdens: no need to worry about hardware provisioning, setup, configuration, replication, software patching, or cluster scaling.
- Offers **encryption at rest**, reducing the complexity of protecting sensitive data.
- Ideal for applications requiring low-latency, high-scale data access.

4.9 Amazon SimpleDB

Definition

Amazon SimpleDB is a **lightweight, highly scalable, and flexible** data storage solution for applications that do **not require a fully relational model**.

Data Model

SimpleDB supports **semi-structured data** based on three concepts:

- **Domains** — equivalent to tables in relational databases.
- **Items** — equivalent to rows.
- **Attributes** — equivalent to columns but with fewer structural constraints.

Key Features

- Fewer constraints on data structure → **improved performance** for querying large quantities of data.
- A **highly available NoSQL data store** — handles database administration automatically.
- Developers store and query data using **web service requests**.

Current Status

SimpleDB is an **older AWS service** that is essentially deprecated in practice:

- It is **not listed** on the main AWS products page.
- Its developer guide **hasn't been updated in years**.
- AWS still runs the service for existing customers, but it is **not actively promoted or developed**.

PART 5: Communication Services

5.1 Overview

Amazon provides facilities for **structuring and facilitating communication** among applications and services within the AWS infrastructure. These fall into two major categories:

1. **Virtual Networking**
2. **Messaging**

5.2 Virtual Networking

Definition

Virtual networking is a collection of services that allow AWS users to control **connectivity to and between compute and storage services**.

Three Main Services:

1. **Amazon VPC** — Infrastructure-level connectivity (private virtual networks)
2. **Amazon Direct Connect** — Infrastructure-level connectivity (dedicated lines)
3. **Amazon Route 53** — Naming/DNS connectivity

Amazon VPC (Virtual Private Cloud)

Definition: Amazon VPC provides flexibility in creating **virtual private networks** within the Amazon infrastructure and beyond.

Features:

- Service providers can use **pre-built templates** for common scenarios or a **fully customizable network** for advanced configurations.

Prepared Templates Include:

- **Public subnets** — accessible from the Internet.
- **Isolated networks** — no Internet access.
- **Private networks with NAT** — private instances access the Internet through Network Address Translation.
- **Hybrid networks** — combining AWS resources with private on-premises resources.

IAM Integration: Connectivity between services (EC2 instances and S3 buckets) can also be controlled using **IAM (Identity Access Management)** service.

Amazon Direct Connect

Definition: Amazon Direct Connect allows AWS users to create **dedicated network connections** between the user's private network and Amazon Direct Connect locations (called **ports**).

Advantages:

- Provides **consistent, predictable performance** compared to standard Internet-based connections.
- Compatible with **EC2, S3, and Amazon VPC**.
- Used in scenarios requiring **high bandwidth** between the Amazon network and the outside world.

Amazon Route 53

Definition: Amazon Route 53 implements **dynamic DNS (Domain Name System) services**, allowing AWS resources to be reached through **custom domain names** rather than the default amazon.com domain.

5.3 Messaging Services

Messaging services connect applications by leveraging AWS capabilities. There are three types:

1. **Amazon SQS** — Simple Queue Service
2. **Amazon SNS** — Simple Notification Service
3. **Amazon SES** — Simple Email Service

Amazon SQS (Simple Queue Service)

Definition: SQS provides a **disconnected (asynchronous) model** for exchanging messages between applications through **message queues** hosted within the AWS infrastructure.

Key Features:

- Users can create an **unlimited number of message queues** using the AWS console or Web service.
- Messages are **securely and redundantly stored** within AWS for a **limited period of time**.

- Only **authorized applications** can access these messages.
- While a message is being read, it is **locked** to prevent other applications from processing it simultaneously (spurious processing prevention).
- The lock **expires after a given period** if not acknowledged.

Use Case: Reliable, decoupled communication between microservices or separate application components.

Amazon SNS (Simple Notification Service)

Definition: Amazon SNS provides a **publish-subscribe (pub/sub) model** for connecting heterogeneous (different types of) applications.

How It Works:

1. AWS users create a **Topic** — a logical access point for messages.
2. Other applications (subscribers) **subscribe** to the topic.
3. Any application can **publish content** to the topic.
4. All subscribers are **automatically notified** when new content is published.

Two Types of Clients:

- **Publishers (Producers):** Send messages/notifications to an SNS topic. No need to specify destination addresses — the topic handles routing automatically.
- **Subscribers (Consumers):** Applications (web servers, email addresses, SQS queues) that receive notifications over supported protocols including **Amazon SQS, email, HTTP, SMS**.

Amazon SES (Simple Email Service)

Definition: Amazon SES provides AWS users with a **scalable email service** built on the AWS infrastructure for sending bulk or transactional emails.

Setup Process:

1. User signs up for SES and provides an **email address**.
2. SES sends a **verification email** to confirm the address.
3. Upon verification, the user gets access to an **SES sandbox** to test the service.

Key Features:

- Can integrate into **any application** for bulk email sending.

- Supports both **transactional emails** (e.g., order confirmations) and **marketing emails** (e.g., newsletters).
- **Pay-per-use** pricing model.
- Provides **detailed email campaign statistics** to help users improve communication effectiveness.

PART 6: Additional Services

6.1 Overview

Beyond compute, storage, and communication, AWS provides additional services for monitoring and payments:

- **Amazon CloudWatch**
- **Amazon Flexible Payment Service (FPS)**

6.2 Amazon CloudWatch

Definition

Amazon CloudWatch is a **comprehensive monitoring and observability service** that provides statistics to help developers **understand and optimize** the behavior of their applications hosted on AWS.

> *"Observe and monitor resources and applications on AWS, on premises, and on other clouds."*
> — Amazon AWS

Data Sources

CloudWatch collects information from several AWS services, including:

- EC2
- S3
- SimpleDB
- CloudFront
- and others.

Key Features

1. Usage Monitoring: Developers can see a **detailed breakdown** of their service usage on AWS, helping them design **more efficient and cost-saving applications**.

2. Pricing: Earlier, CloudWatch required a subscription. Now it is **available for free** to all AWS users.

3. Dashboards:

- The CloudWatch home page **automatically displays metrics** about every AWS service in use.
- Users can create **customized dashboards** to display metrics for specific applications or custom collections.

4. Alarms:

- Users can create **alarms** that watch specific metrics.
- When a **threshold is breached**, CloudWatch can:
  - **Send notifications** to the user.
  - **Automatically change resources** being monitored (e.g., launch or terminate EC2 instances).

Example Use Cases:

- Monitor **CPU usage and disk reads/writes** of EC2 instances → determine if additional instances are needed for handling increased load.
- Identify **under-used instances** and stop them → **save money**.

6.3 Amazon Flexible Payment Service (FPS)

Definition

Amazon FPS is a **payment service** that allows AWS users to leverage Amazon's billing infrastructure to **sell goods and services** to other AWS users.

> *"Amazon FPS is the first payments service designed from the ground up specifically for developers. The set of web service APIs allows the movement of money between any two entities, humans or computers."* — Amazon

Problem It Solves

Traditionally, developers were limited in managing payments. If a developer needed to:

- Charge customers at a **certain frequency**
- Execute a transaction at a **specific time**
- **Combine smaller payments** into one transaction
- **Charge a commission** on a transaction between two customers

...they would need to build their own payment infrastructure and establish processor relationships, which is **difficult, expensive, time-consuming, and operationally risky**.

How FPS Helps

Payment Instructions: A capability called **"Payment Instructions"** gives developers the flexibility to build **multiple custom charging models**.

Examples of Charging Models:

- Charge customers in **small increments** until a cumulative balance reaches a limit.
- Pay a **percentage of a digital transaction as a royalty**.
- Earn a **commission on a marketplace transaction**.
- Allow **one customer to pay for another customer** and limit their usage to a specific amount.

Key Characteristics

- 100% focused on the **needs of developers**.
- APIs allow movement of money between **any two entities** (human or automated processes).
- Eliminates the need for custom payment infrastructure.

Summary Table — AWS Services at a Glance

Category	Service	Purpose
Compute	EC2	Virtual machine instances for running applications
Compute	AMI	Templates for creating EC2 instances
Compute	Elastic MapReduce	MapReduce (big data processing) on Hadoop
Storage	S3	Distributed object storage
Storage	EBS	Persistent block storage for EC2
Storage	ElastiCache	In-memory caching clusters
Database	Preconfigured AMIs	EC2-hosted databases (user-managed)
Database	RDS	Fully managed relational databases
Database	DynamoDB	Fully managed NoSQL database

Category	Service	Purpose
Database	SimpleDB	Lightweight semi-structured data store
Networking	VPC	Virtual private networks
Networking	Direct Connect	Dedicated, high-bandwidth connections
Networking	Route 53	DNS and domain name management
Messaging	SQS	Asynchronous message queuing
Messaging	SNS	Publish-subscribe notifications
Messaging	SES	Bulk/transactional email service
Monitoring	CloudWatch	Usage metrics, alarms, dashboards
Payment	FPS	Developer-friendly payment APIs

End of Module 3.2 Study Notes